

Transfer Learning with Pretrained Large Language Models

AdvNLE Seminar 8

Dr Julie Weeds, Spring 2026



Previously ...

- Language models
- Distributional representations of meaning
- Composition
- Contextualised word embeddings
 - ELMo
 - transformers
 - BERT
- Pre-training Large Language Models

This week

- Transfer learning
- Fine-tuning BERT-based models for
 - Text / sequence classification
 - sequence labelling
- The BERT family
- More distant relatives
- Pattern Exploiting Training (PET) - Schick and Schütze (2021)

Discussion

- You want to build a classifier which can detect racially offensive language in tweets.
- You have collected 500 examples of racially offensive tweets, all posted on X in the last 7 days.
- What are your options in terms of building a classifier?
- What are the advantages and disadvantages of each?
- Are there any other considerations?

Transfer Learning through Fine-tuning

- **Transfer learning:** acquiring knowledge from one task or domain and then applying (transferring) it to solve a new task
- BERT-based models acquire knowledge about language through **pre-training** (masked language model prediction and next sentence prediction) on large unannotated corpora
- **Fine-tuning** is the process of transferring this knowledge to a specific task e.g., is a tweet racially offensive or not?

Fine-tuning

- Fine-tuning **uses labelled data** from the application to train **additional application-specific parameters**
- Application-specific parameters could be a single layer neural network on top of the BERT architecture
- Fine-tuning might
 - freeze the pre-trained language model parameters
 - allow updates to be made to some or all of the pre-trained language model parameters

Text classification with BERT

- E.g., Is a tweet **racially offensive** or **not**?
- [CLS] token is used to stand for the entire sequence
- [CLS] is part of the vocabulary and must be pre-pended to all input sequences during pre-training and fine-tuning
- [CLS] is the input to a classifier head, e.g., logistic regression, which makes relevant decision

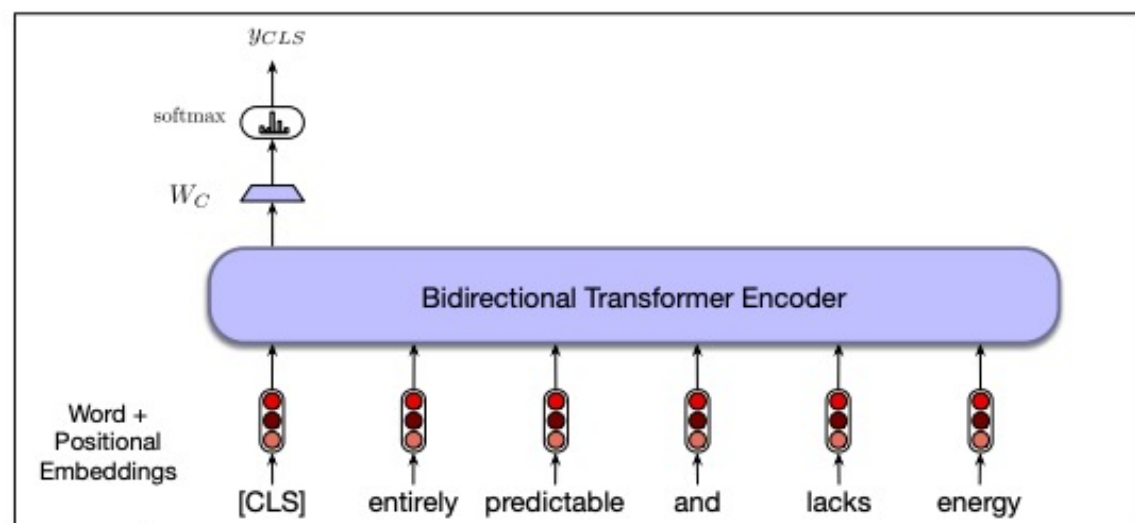


Figure 11.8 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

Figure taken from Jurafsky and Martin (Section 11.3.1)

Text classification with BERT (2)

- learn a set of weights W_C which maps output vector for [CLS] to a set of scores for the possible classes
- Pass input text through pre-trained language model to generate y_{CLS} , multiply it by W_C and then pass this through softmax

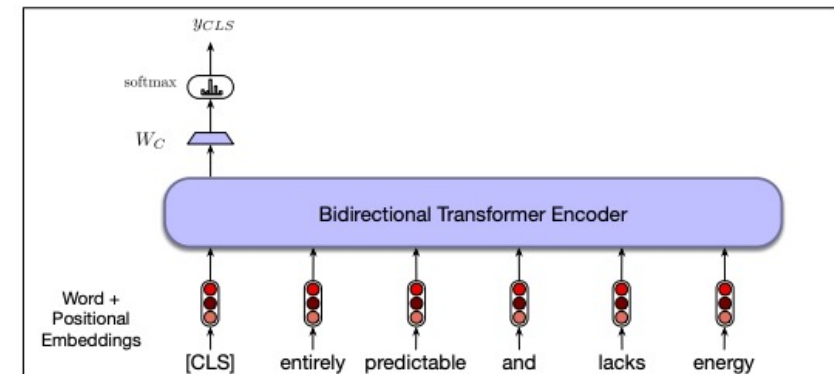


Figure 11.8 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

$$y = \text{softmax}(W_C y_{CLS})$$

- Fine-tuning requires input sequences labelled with appropriate class
- Cross-entropy loss between softmax output and correct answer drives backpropagation
- Weights can be updated just for W_C (pre-trained language model is frozen) or in the pre-trained language model as well

Code for BERTclassifier (from lab 9)

```
] : #now we need to put a simple classification layer on top of BERT

from torch import nn
from transformers import BertModel

class BertClassifier(nn.Module):

    def __init__(self, dropout=0.5, num_classes=2):

        super(BertClassifier, self).__init__()

        self.bert=BertModel.from_pretrained('bert-base-uncased')
        self.dropout=nn.Dropout(dropout)
        self.linear=nn.Linear(768, num_classes)
        self.relu=nn.ReLU()

    def forward(self, input_id, mask):

        last_hidden_layer, pooled_output = self.bert(input_ids=input_id, attention_mask=mask, return_dict=False)
        dropout_output=self.dropout(pooled_output)
        linear_output=self.linear(dropout_output)
        final_layer=self.relu(linear_output)

        return final_layer
```

Freezing layers

```
In [ ]: model=BERTClassifier(num_classes=len(labels.keys()))  
  
#this will freeze the pre-trained BERT model and just make the classification head trainable  
#can speed things up and avoid "catastrophic forgetting" / overfitting on task-specific data  
model.bert.requires_grad_(False)
```

class label

LR
classification

rocks

Softmax
over vocab

BERT encoder layer (multi-head attention + feedforward)

BERT encoder layer (multi-head attention + feedforward)

Input embedding =
Token embedding +
positional embedding

[CLS]

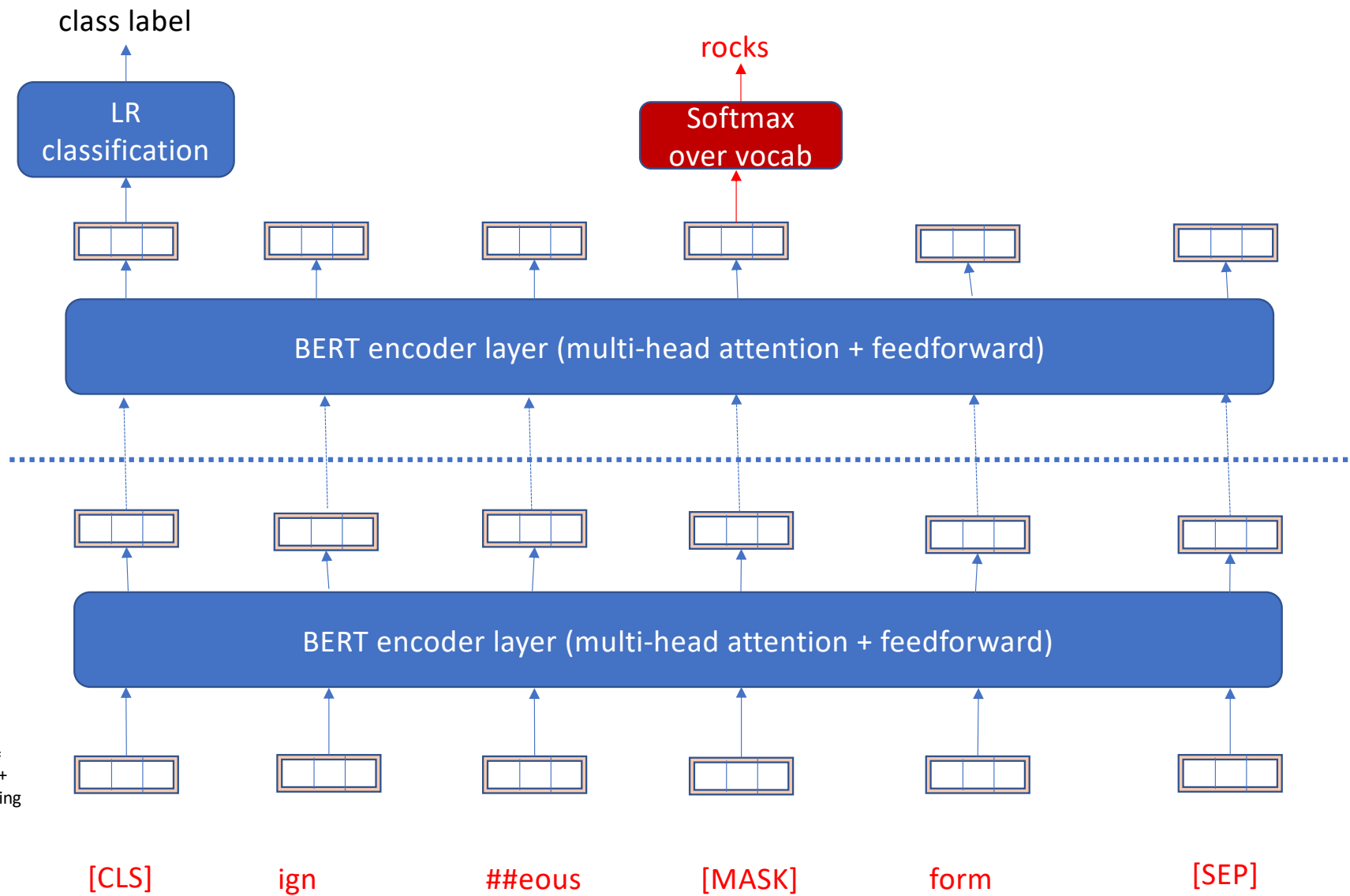
ign

##eours

[MASK]

form

[SEP]



Pairwise Sequence Classification

- Do two sentences entail or contradict each other?
 1. “I’m confused”
 2. “It is not completely clear to me”
 3. “I’m confused about transfer learning”
 4. “Everything is really clear to me”
- Concatenate the sentences, separated by the [SEP] token
- Proceed as for single sequence classification
- OR use something like SBERT. ***What’s the advantages of each approach?***

Sequence Labelling with BERT

- E.g., POS tagging or NER
- Simplest approach is to pass each output from BERT to a simple classifier
- Or pass it to a CRF which considers tag-level transitions as well
- Complications can arise from subword tokenizations
 - simplest approach is to use the first subword token from each word

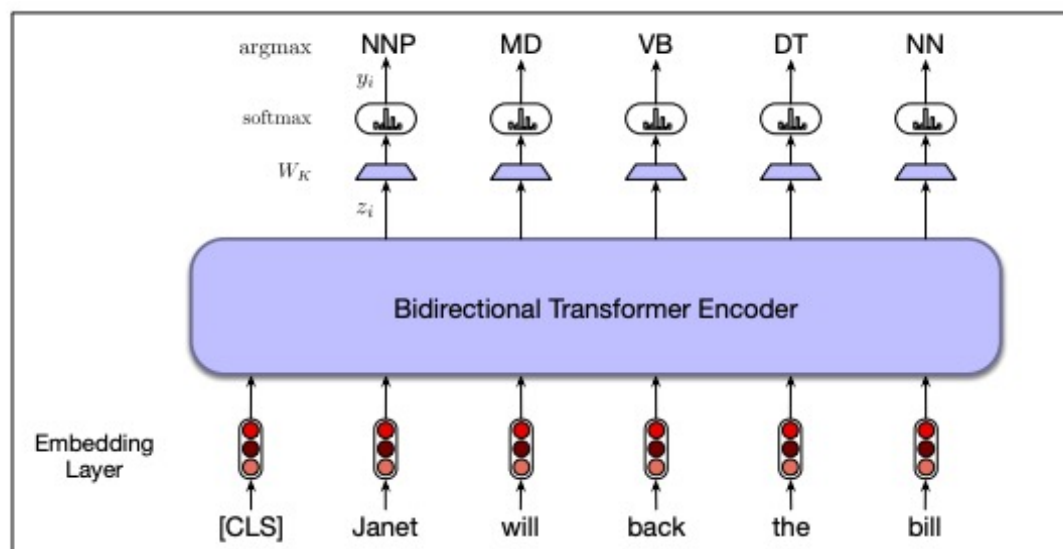


Figure 11.9 Sequence labeling for part-of-speech tagging with a bidirectional transformer encoder. The output vector for each input token is passed to a simple k-way classifier.

Lab 9b

- Bonus lab!
- Have a go at fine-tuning BERT for Named Entity Recognition

What makes each of these members of the BERT Family distinctive?

- RoBERTa
- ALBERT
- DistilBERT
- SciBERT
- MBert
- and more
 - StructBERT
 - AraBERT
 - DeBERTa

Seminar Reading

- Schick and Schütze (2021): Exploiting Cloze Questions for Few Shot Text Classification and Natural Language Inference

Schick and Schütze (2021) introduce Pattern-Exploiting Training (PET), a semi-supervised training procedure that reformulates input examples as cloze-style phrases to help language models understand a given task. Once you have read the paper, consider the following questions.

1. What do you understand by the term few-shot learning? Why is it important /challenging in NLP?
2. What is a Cloze question or Cloze-style phrase?
3. What is pattern-verbalizer pair (PVP)? Give some examples of your own that might be used for different tasks e.g., sentiment classification and paraphrasing
4. How are the PVPs used in training and inference?
5. What is catastrophic forgetting and how is it avoided?
6. How are different PVPs used to create a final model?
7. What is iPET?
8. What experiments did the authors carry out to demonstrate the effectiveness of their approach? What do you think of the results?
9. What does the analysis in section 5 tell us?
10. What are the main conclusions? Are you convinced? Would you use this approach?

References

Timo Schick and Hinrich Schütze. 2021. Exploiting cloze questions for few shot text classification and natural language inference. In *Proceedings of the European Chapter of the Association for Computational Linguistics 2021*.

1. What do you understand by the term few-shot learning? Why is it important/ challenging in NLP?

1. What do you understand by the term few-shot learning? Why is it important/ challenging in NLP?

Learning from examples is the predominant approach for many NLP tasks: A model is trained on a set of labeled examples from which it then generalizes to unseen data. Due to the vast number of languages, domains and tasks and the cost of annotating data, it is common in real-world uses of NLP to have only a small number of labeled examples, making *few-shot learning* a highly important research area. Unfortunately, applying standard supervised learning to small training sets often performs poorly; many problems are difficult to grasp from just looking at a few examples. For instance, assume we are given the following pieces of text:

- T_1 : This was the best pizza I've ever had.
- T_2 : You can get better sushi for half the price.
- T_3 : Pizza was average. Not worth the price.



The label for T_1 is L
The label for T_2 is L'
What is the label for T_3 ?

2. What is a Cloze question or Close-style phrase?

2. What is a Cloze question or Cloze-style phrase?

Cloze test

🌐 9 languages ▾

Article [Talk](#)

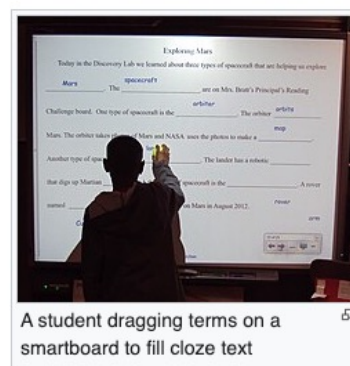
Read [Edit](#) [View history](#) [Tools](#) ▾

From Wikipedia, the free encyclopedia

A **cloze test** (also **cloze deletion test** or **occlusion test**) is an exercise, test, or assessment in which a portion of text is **masked** and the participant is asked to fill in the masked portion of text. Cloze tests require the ability to understand the context and **vocabulary** in order to identify the correct language or part of speech that belongs in the deleted passages. This exercise is commonly administered for the assessment of native and second **language learning** and instruction.

The word *cloze* is derived from *closure* in **Gestalt theory**. The exercise was first described by Wilson L. Taylor in 1953.^[1]

Words may be deleted from the text in question either mechanically (every nth word) or selectively, depending on exactly what aspect it is intended to test for. The methodology is the subject of extensive academic literature;^[2] nonetheless, teachers commonly devise ad hoc tests.



A student dragging terms on a smartboard to fill cloze text

Its just
another name
for Masked
Language
modelling!

Examples [\[edit \]](#)

A language teacher may give the following passage to students:

Today, I went to the _____ and bought some milk and eggs. I knew it was going to rain, but I forgot to take my _____, and ended up getting wet on the way.

3. What is a pattern-verbalizer pair (PVP)? Give some examples of your own that might be used for different tasks e.g., sentiment classification and paraphrasing

3. What is a pattern-verbalizer pair (PVP)? Give some examples of your own that might be used for different tasks e.g., sentiment classification and paraphrasing

sentences). We define a *pattern* to be a function P that takes $\mathbf{x}$ as input and outputs a phrase or sentence $P(\mathbf{x}) \in V^*$ that contains exactly one mask token, i.e., its output can be viewed as a cloze question. Furthermore, we define a *verbalizer* as an injective function $v : \mathcal{L} \rightarrow V$ that maps each label to a word from M 's vocabulary. We refer to (P, v) as a *pattern-verbalizer pair* (PVP).

3. What is a pattern-verbalizer pair (PVP)? Give some examples of your own that might be used for different tasks e.g., sentiment classification and paraphrasing

- Sentiment classification

- $P(x)$ = "This was the best pizza I've ever had. Therefore, I _____ this restaurant"

- $V(y) = \{\text{liked, disliked}\}$

- Paraphrasing

- $P(x)$ = "Keir Starmer is the British prime minister? _____> The head of the UK government is Keir Starmer"

- $V(y) = \{\text{Yes, No}\}$

4. How are PVPs used in training and inference?

4. How are PVPs used in training and inference?

- Section 3.1
 - M is a masked language model
 - Find the score for each label given each training input by finding the unnormalized score that M assigns to each verbalizer at the masked position in the input
 - Obtain a probability distribution over labels using softmax
 - Use cross-entropy loss between predicted distribution and true distribution as loss for finetuning M for p.

5. What is catastrophic forgetting and how is it avoided?

5. What is catastrophic forgetting and how is it avoided?

- Section 3.2
- Overfitting to the fine-tuning / secondary data
- Avoid by using language modelling as an auxiliary task
- Use the unlabelled examples which have been converted into $P(x)$
- Randomly mask words (but do not ask M to predict anything for the verbalizer slot)

$$L = (1 - \alpha) \cdot L_{\text{CE}} + \alpha \cdot L_{\text{MLM}}$$

6. How are different PVPs used to create a final model?

6. How are different PVPs used to create a final model?

- Section 3.3
- Fine-tune a separate language model M_p for each PVP based on the small set of annotated training examples T
- Use the ensemble of models to annotate examples from the unlabelled set D
- This provides a “soft” label for each of the unlabelled examples in D
- Finetune a new model with a standard sequence classification head on these soft-labelled examples.

7. What is iPET?

7. What is iPET?

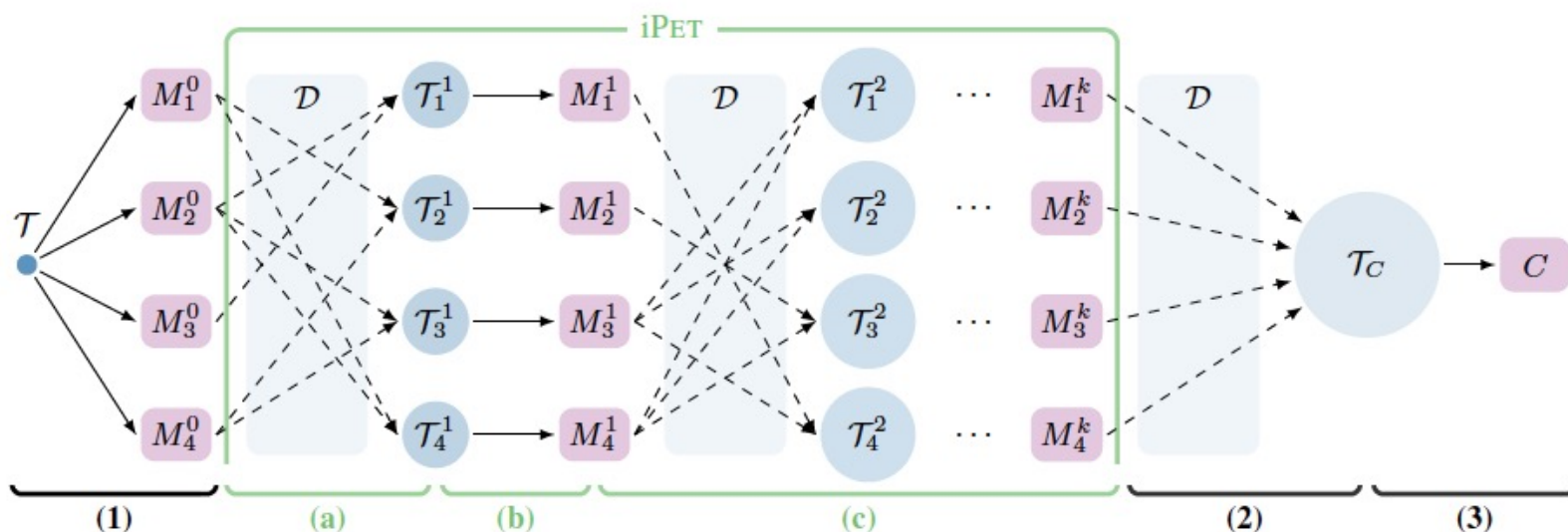


Figure 2: Schematic representation of PET (1-3) and iPET (a-c). (1) The initial training set is used to finetune an ensemble of PLMs. (a) For each model, a random subset of other models generates a new training set by labeling examples from $\mathcal{D}$. (b) A new set of PET models is trained using the larger, model-specific datasets. (c) The previous two steps are repeated k times, each time increasing the size of the generated training sets by a factor of d . (2) The final set of models is used to create a soft-labeled dataset $\mathcal{T}_C$. (3) A classifier C is trained on this dataset.

8. What experiments do the authors carry out to demonstrate the effectiveness of their approach? What do you think of the results?

8. What experiments do the authors carry out to demonstrate the effectiveness of their approach? What do you think of the results?

We evaluate PET on four English datasets: Yelp Reviews, AG's News, Yahoo Questions (Zhang et al., 2015) and MNLI (Williams et al., 2018). Additionally, we use x-stance (Vamvas and Senrich, 2020) to investigate how well PET works for other languages. For all experiments on English, we use RoBERTa large (Liu et al., 2019) as language model; for x-stance, we use XLM-R (Conneau et al., 2020). We investigate the performance of PET and all baselines for different training set sizes; each model is trained three times using different seeds and average results are reported.

8. What experiments do the authors carry out to demonstrate the effectiveness of their approach? What do you think of the results?

Line	Examples	Method	Yelp	AG's	Yahoo	MNLI (m/mm)
1	$ \mathcal{T} = 0$	unsupervised (avg)	33.8 $\pm$ 9.6	69.5 $\pm$ 7.2	44.0 $\pm$ 9.1	39.1 $\pm$ 4.3 / 39.8 $\pm$ 5.1
2		unsupervised (max)	40.8 $\pm$ 0.0	79.4 $\pm$ 0.0	56.4 $\pm$ 0.0	43.8 $\pm$ 0.0 / 45.0 $\pm$ 0.0
3		iPET	56.7 $\pm$ 0.2	87.5 $\pm$ 0.1	70.7 $\pm$ 0.1	53.6 $\pm$ 0.1 / 54.2 $\pm$ 0.1
4	$ \mathcal{T} = 10$	supervised	21.1 $\pm$ 1.6	25.0 $\pm$ 0.1	10.1 $\pm$ 0.1	34.2 $\pm$ 2.1 / 34.1 $\pm$ 2.0
5		PET	52.9 $\pm$ 0.1	87.5 $\pm$ 0.0	63.8 $\pm$ 0.2	41.8 $\pm$ 0.1 / 41.5 $\pm$ 0.2
6		iPET	57.6 $\pm$ 0.0	89.3 $\pm$ 0.1	70.7 $\pm$ 0.1	43.2 $\pm$ 0.0 / 45.7 $\pm$ 0.1
7	$ \mathcal{T} = 50$	supervised	44.8 $\pm$ 2.7	82.1 $\pm$ 2.5	52.5 $\pm$ 3.1	45.6 $\pm$ 1.8 / 47.6 $\pm$ 2.4
8		PET	60.0 $\pm$ 0.1	86.3 $\pm$ 0.0	66.2 $\pm$ 0.1	63.9 $\pm$ 0.0 / 64.2 $\pm$ 0.0
9		iPET	60.7 $\pm$ 0.1	88.4 $\pm$ 0.1	69.7 $\pm$ 0.0	67.4 $\pm$ 0.3 / 68.3 $\pm$ 0.3
10	$ \mathcal{T} = 100$	supervised	53.0 $\pm$ 3.1	86.0 $\pm$ 0.7	62.9 $\pm$ 0.9	47.9 $\pm$ 2.8 / 51.2 $\pm$ 2.6
11		PET	61.9 $\pm$ 0.0	88.3 $\pm$ 0.1	69.2 $\pm$ 0.0	74.7 $\pm$ 0.3 / 75.9 $\pm$ 0.4
12		iPET	62.9 $\pm$ 0.0	89.6 $\pm$ 0.1	71.2 $\pm$ 0.1	78.4 $\pm$ 0.7 / 78.6 $\pm$ 0.5
13	$ \mathcal{T} = 1000$	supervised	63.0 $\pm$ 0.5	86.9 $\pm$ 0.4	70.5 $\pm$ 0.3	73.1 $\pm$ 0.2 / 74.8 $\pm$ 0.3
14		PET	64.8 $\pm$ 0.1	86.9 $\pm$ 0.2	72.7 $\pm$ 0.0	85.3 $\pm$ 0.2 / 85.5 $\pm$ 0.4

Table 1: Average accuracy and standard deviation for RoBERTa (large) on Yelp, AG's News, Yahoo and MNLI (m:matched/mm:mismatched) for five training set sizes $|\mathcal{T}|$.

9. What does the analysis in section 5 tell us?

9. What does the analysis in section 5 tell us?

- Large gap in performance for different PVPs but PET compensates for this
- Distillation helps
- Auxiliary language modelling helps more for small number of training examples
- More model generations seems to help iPET
- In-domain pretraining helps PET and other models

10. What are the main conclusions? Are you convinced? Would you use this approach?

10. What are the main conclusions? Are you convinced? Would you use this approach?

6 Conclusion

We have shown that providing task descriptions to pretrained language models can be combined with standard supervised training. Our proposed method, PET, consists of defining pairs of cloze question patterns and verbalizers that help leverage the knowledge contained within pretrained language models for downstream tasks. We finetune models for all pattern-verbalizer pairs and use them to create large annotated datasets on which standard classifiers can be trained. When the initial amount of training data is limited, PET gives large improvements over standard supervised training and strong semi-supervised approaches.